Does Write-Side State Adjudication Survive Small Models?

Reproducing and Stress-Testing STALE and CUPMem

Project Log & Technical Record #2
First API Runs: Key Setup, Rate Limits, and the B0 Smoke Test

Date: 3 October 2026
Follows: Record #1 (27 September 2026, notes/week1_progress_report.pdf)
Workspace: `C:\research paper`
Coding assistant: Claude Code (Claude Opus 5.5)

Contents

Executive summary 4

Background: the paper and this project 5

B1. The paper in one paragraph . . . 5

B2. The problem: implicit conflicts . . . 5

B3. How the benchmark is built and scored . . . 5

B4. What the paper found . . . 6

B5. CUPMem: the proposed fix . . . 7

B6. This project: what we test and why . . . 7

Part I: What changed 9

1. Change summary . . . 9

2. Step-by-step log . . . 9

3. The rate-limit analysis . . . 11

4. Code changes in detail . . . 12

Part II: First results 14

5. Smoke-test results . . . 14

Part III: Housekeeping and next steps 18

6. Security and key handling . . . 18

7. Updated status . . . 18

8. Open decisions and next steps . . . 18

Appendix: Current source of changed files 19

A.1 src/stale_repro/llm/gateway.py (291 lines) . . . 20

A.2 src/stale_repro/runner.py (132 lines) . . . 24

A.3 scripts/run_experiment.py (39 lines) . . . 25

A.4 configs/b0_gpt4omini_dev.yaml (23 lines) . . . 26

A.5 tests/test_gateway.py (108 lines) 27

A.6 Smoke-test call log (runs/b0_gpt4omini_dev_n2/llm_calls.jsonl) 28

Executive summary

Record #1 ended with a tested pipeline that had never called a real model. This session connected it to the OpenAI API and ran the first real experiment: a smoke test of the B0 full-context baseline (GPT-4o-mini) on four STALE instances (two per conflict type).

Outcome	Detail
API access verified	The OpenAI key works. It is stored only in the gitignored .env file.
Paper's model still available	gpt-4o-mini resolves to the snapshot gpt-4o-mini-2024-07-18, the same model the paper used. It is now pinned in the config, so no successor-model substitution is needed for E1.
Rate-limit problem found and fixed	The account limit (200K tokens/minute) admits only about one 148K-token B0 call per minute. The first attempt lost 2 of 4 instances to HTTP 429 errors. Fix: a client-side token-rate limiter plus smarter retries in the gateway.
Truncation parity confirmed	Our token estimate matched OpenAI's billed prompt tokens to within 2 tokens per request.
Paper's failure pattern observed	Premise Resistance answers accepted outdated premises, as the paper reports for GPT-4o-mini.
Judge stage deferred	No Gemini key yet. Added a --no-judge mode; answers are cached and can be judged later at no extra generation cost.
Tests	25 passing (2 new for the rate limiter).
Smoke-test status at build time	4 of 4 instances answered; 12 real API calls; 1,485,491 input + 4,429 output tokens; cost about $0.23.

Key practical finding. At the current account tier the rate limit, not money, is the bottleneck for full-context runs:
- 50-instance dev run: about 2.5 hours;
- full 400-instance B0: about 20 hours.

The Batch API or a higher usage tier would remove this bottleneck (Section 3.3). CUPMem runs are less affected because they make many small calls rather than a few huge ones.

Readers new to the project should start with the Background part that follows. It explains the paper, the benchmark, the proposed method and what this project tests, so that the results in Part II can be read in context.

Background: the paper and this project

This part summarises the paper being reproduced and explains how this project relates to it. Record #1 (Part I) has more dataset examples and the full CUPMem state schema.

B1. The paper in one paragraph

STALE: Can LLM Agents Know When Their Memories Are No Longer Valid? (Chao, Bai, Sheng, Li and Sun; Wuhan University, CUHK and HKUST; arXiv:2605.06527, May 2026). LLM assistants are increasingly expected to remember users across many sessions. Existing memory benchmarks mostly test whether a stored fact can be *retrieved*. They do not test whether the assistant notices that a stored fact has become *wrong*. The paper names this failure mode the **implicit conflict**. It contributes three things:

- **STALE**, a benchmark of 400 expert-validated implicit-conflict scenarios (1,200 queries) covering more than 100 everyday topics, with histories of up to about 150K tokens;
- an **evaluation of 15 systems** (frontier LLMs, open models and memory frameworks), which all perform poorly. The best system reaches only 55.2% overall accuracy;
- **CUPMem**, a prototype memory system that judges at write time whether old memories are still valid. With the same GPT-4o-mini backbone it raises overall accuracy from 8.7% to 68.0%.

B2. The problem: implicit conflicts

People rarely announce that an old fact has stopped being true. A user who once said "I ride my bike to work every day" will later mention a broken leg, not "please forget that I cycle". The new observation *invalidates* the old memory without ever *negating* it. Detecting this requires commonsense inference rather than string matching. The paper formalises the idea with two axioms:

- **Belief incompatibility:** the old observation supports a belief that cannot hold together with what the new observation implies.
- **Non-explicit invalidation:** nothing in between states the change outright.

Conflict type	What changes	Example
Type I (co-referential)	The same attribute gets a new value, which is implied but never named.	"Based in Seattle", later: setting up utilities at a new apartment in Austin.
Type II (propagated)	A change in attribute A makes a *different* attribute B invalid. Neither B nor the A-to-B link is mentioned.	"Cycles to work daily", later: a broken leg. The commute memory is now unsafe.

B3. How the benchmark is built and scored

B3.1 Construction pipeline (paper Section 3.4)

Step	What happens
1. Base state	Sample an attribute from a topic ontology. An LLM writes a persona and the old observation m_{old}, which clearly supports one value.
2. Adversarial conflict	A "Logic Attacker" LLM writes the new observation m_{new} after a time gap. It implies a new value (Type I) or changes an upstream attribute (Type II), without naming the target.
3. Quality control	A strict LLM judge checks plausibility, a real state-level conflict and implicitness, and rejects pairs with obvious cues. Failed pairs are regenerated. Human experts then review and edit every instance.
4. Haystack	Each observation is wrapped in a natural 8-12 turn dialogue session. The two sessions are embedded in a 50-session chronological history padded with filtered LongMemEval distractor sessions.

In the released data, the old evidence always sits in sessions 0-14 and the new evidence in sessions 30-39, about 27 sessions apart on average. A serialized history averages about 152K tokens, so every instance exceeds GPT-4o-mini's 128K context window (Record #1).

B3.2 Three probes per scenario

Probe	Tests whether the model...	Example (cycling scenario)
SR, State Resolution	recognises, when asked directly, that the old belief is outdated	"Based on the conversation history, does the user still commute by cycling?"
PR, Premise Resistance	refuses a request that assumes the old state is still true	"Since the user rides a bike every day, can you create a maintenance plan?"
IPA, Implicit Policy Adaptation	applies the new state unprompted in an ordinary request	"Can you suggest a commute plan for this week?"

Each probe is a separate model call, so one answer cannot leak into another. An LLM judge (Gemini-3.1-flash-lite) sees the old state, the new state and a hidden explanation, and marks each answer pass or fail. It agreed with human labels 95.8% of the time (Cohen's kappa 0.915). **Overall** accuracy is the unweighted mean of the six (type x probe) cells.

B4. What the paper found

System (selection)	T1 SR	T1 PR	T1 IPA	T2 SR	T2 PR	T2 IPA	Overall
GPT-4o-mini* (plain, full context)	30.0	0.0	11.0	9.5	0.0	1.5	8.7
Gemini-3.1-pro (best plain LLM)	92.0	30.0	71.0	69.0	14.0	55.0	55.2
Qwen3.5-9B	36.0	1.0	21.5	21.5	0.0	7.5	14.6
LightMem (best framework, GPT-4o-mini)	52.5	1.0	23.5	21.5	0.5	7.5	17.8
CUPMem (GPT-4o-mini)	91.0	**78.0**	32.0	**89.0**	**75.0**	43.0	**68.0**

* evidence-preserving truncation: both evidence sessions are kept and only distractors are removed. The **GPT-4o-mini row is exactly what our B0 baseline reproduces**, and Part II is its first test.

B4.1 Three headline findings

- **Recognising is not applying.** SR and IPA scores diverge. Qwen3.5-27B scores 76.0% on Type I SR but 39.0% on Type I IPA.
- **Premises are followed blindly.** PR is the weakest probe for almost every system. Gemini-3.1-pro falls from 92.0% (SR) to 30.0% (PR) on Type I.
- **Propagated conflicts are harder.** Type II is below Type I on almost every cell.

B4.2 The paper's explanations

- **Plain LLMs do not reconcile sessions (attention analysis, Qwen3.5-9B and 27B).** The query attends to both evidence sessions well above a positional-noise baseline, but the new session barely attends to the old one. Models route between the two sessions per query rather than updating a belief. Correct IPA answers put relatively more attention on the new session, especially in middle layers.
- **Memory frameworks see the evidence but do not act on it (the "current-state adjudication gap").** In LightMem, the new evidence is retrieved in 77.5% of SR/PR cases. When it is written, the old memory appears in its top-3 neighbours 60.5% of the time, yet only 3.3% of those old entries are marked for update. Old and new memories end up side by side and nothing decides between them. In the paper's words, "visibility does not imply authority".

B5. CUPMem: the proposed fix

CUPMem (Current-state Updating and Propagation-aware Memory) turns memory into explicit state tracking. Its central idea is to resolve conflicts **when new evidence is written**, not leave them for the answering model at query time. Memories are stored in a fixed two-level schema (10 domains such as location_and_living or health_and_mobility, each with typed slots), and each memory is marked active or stale.

Stage	What it does	Which failure it targets
1. Write-side adjudication	Extracts state-update candidates from each new session. An LLM adjudicator then labels every related old memory KEEP, STALE, REPLACE or UNKNOWN. Stale memories are archived, not deleted. A slot that is no longer safe but has no replacement becomes UNKNOWN_CURRENT.	The adjudication gap: new evidence gains the authority to retire old memories.
2. Propagation-aware search	Chooses which old memories to re-check: the union of the *Direct* slots touched, the *Affected* regions an LLM predicts may change (e.g. health affects commute), and a bounded embedding-similarity fallback $Global_k$.	Type II: invalidations in a different slot.
3. Constrained readout	At query time, analyses the query's presupposed states. A verifier labels the premise SUPPORTED, OUTDATED or UNRESOLVED. Only active memories ground the answer, and outdated premises are blocked.	PR: answering on a false premise.

This design explains why CUPMem's largest gains are on PR (78.0% and 75.0% versus near zero for the baselines). The paper reports a cost of about $0.37 per instance on GPT-4o-mini. Its LLM calls go to an OpenAI-compatible endpoint, and it uses a local embedding model (all-MiniLM-L6-v2).

B6. This project: what we test and why

Working title: **"Does Write-Side State Adjudication Survive Small Models? Reproducing and Stress-Testing STALE and CUPMem."** CUPMem's 68.0% rests on many LLM judgements made at write time (extraction, affected-region prediction, adjudication), all by GPT-4o-mini. The project asks whether the result holds up, whether it survives cheaper adjudicators, and where it breaks.

RQ	Question	How we answer it
RQ1	Do the paper's numbers reproduce?	Three-way comparison: the paper's figures, the official released code re-run by us, and our independent reimplementation from the paper's description.
RQ2	Does CUPMem still work with 3-9B open models as the adjudicator?	Swap the write-side model for small open models through the same gateway. Malformed JSON outputs are counted as data, not hidden.
RQ3	Which stage fails first?	Per-stage traces: extraction, then candidate search, then adjudication, then readout.
RQ4	What does each component contribute, and at what cost?	Ablate each stage; per-stage token, latency and dollar accounting.
RQ5	Can it handle two-hop cascades (A changes B, which changes C)?	A small new two-hop extension of the data.

B6.1 Open question with the largest impact on RQ1

The official CUPMem runner defaults to `--session-mode relevant_only`. In that mode it ingests only the two gold evidence sessions and skips the 48 distractors that every baseline must process. The paper does not say which mode produced 68.0%. Its $0.37-per-instance cost suggests the full 50-session mode, but this has to be measured. We will run both modes and report `relevant_only` as an explicit oracle-retrieval setting.

B6.2 Experiment ladder and where this record fits

Stage	What	Status
B0 baseline	GPT-4o-mini over the full truncated history (the paper's 8.7% row)	**This record:** first real API runs; 4-instance smoke test answered
Judge	Gemini-3.1-flash-lite with the official prompt	Code ready; waiting for an API key
E1	Full 400-instance reproduction of B0 and CUPMem (paper Table 2 rows)	Next, after a 50-instance dev run
Official CUPMem	Wrap and run the released code in both session modes	To do
Independent CUPMem	Reimplementation from the paper and Appendix F	To do
RQ2-RQ5	Small adjudicators, stage attribution, ablations, two-hop cascades	Later weeks

The project follows a 13-week plan that began on 28 September 2026. The aim is an arXiv preprint around late December 2026, targeting the ML Reproducibility Challenge (TMLR) or an ACL/EMNLP workshop. Every deviation from the paper or the official code is recorded in `notes/reproducibility_log.md`.

Part I: What changed

1. Change summary

File	Change	Why
.env (new, gitignored)	Holds OPENAI_API_KEY; empty slots for Gemini and open-model keys	Keys never enter code, configs or git
src/stale_repro/llm/gateway.py	Added TPMLimiter, estimate_request_tokens, rate-limit-aware retry loop; tpm read from config; SDK retries lowered 5 to 2	Survive a 200K TPM limit with ~148K-token requests
src/stale_repro/runner.py	Answer-only mode: if the config has no judge, stop after answering and write a partial summary	Run generation before the Gemini key exists
scripts/run_experiment.py	New --no-judge flag; load .env by explicit path; prints whichever summary keys exist	Answer-only runs; robust .env loading
configs/b0_gpt4omini_dev.yaml	Model pinned to gpt-4o-mini-2024-07-18; tpm: 190000	Exact model identity; stay under limit with headroom
tests/test_gateway.py	2 new tests: limiter waits when window is full; OpenAI-style token estimate	Cover new code
notes/reproducibility_log.md	New section "2026-10-03: First API runs"	Keep the running record complete
runs/b0_gpt4omini_dev_n2/	First run outputs (answers, call log, config)	Smoke test
cache/llm_cache.sqlite	First cached responses	Reruns and later judging cost nothing

2. Step-by-step log

Step 1: Store the API key safely

Why: The key must be available to the code without ever being committed or printed.

```
Write .env:
  OPENAI_API_KEY=sk-proj-...   (value not reproduced here)
  GEMINI_API_KEY=
  OPEN_MODELS_API_KEY=
  VLLM_API_KEY=EMPTY
```

Result: .env created. It is listed in .gitignore and is never echoed in command output.

Step 2: Verify the key and check model availability

Why: Record #1 flagged the risk that GPT-4o-mini might be retired. Confirm before spending.

```
python - <<EOF
  load_dotenv('.env'); c = OpenAI()
  ids = sorted(m.id for m in c.models.list())
  print gpt-4o-mini family
  c.chat.completions.create(model='gpt-4o-mini', messages=[...'Reply with OK'], max_tokens=5)
EOF
```

Result: 133 models visible. gpt-4o-mini and gpt-4o-mini-2024-07-18 are listed. The test call returned "OK" (10 input, 1 output tokens), served by **gpt-4o-mini-2024-07-18**.

Decision: Pin the snapshot id in the config so the experiment is tied to one exact model version.

Note: the first attempt failed with an AssertionError inside python-dotenv's find_dotenv(). This happens when code is piped through stdin, because there is no calling file to search from. Passing the path explicitly (load_dotenv('.env')) fixed it, and the CLI now does the same.

Step 3: Add an answer-only mode

Why: The judge needs a Gemini key that is not available yet. Generation should not wait for it.

```
runner.py:   if not cfg.get('judge'): write {n_answered, judged: False, usage} and return
run_experiment.py:   --no-judge  ->  cfg['judge'] = None
```

Result: Answers are written and cached. Rerunning later with a judge block judges them without regenerating.

Step 4: Pin the model and rerun the tests

Why: Confirm nothing broke.

```
sed: model: gpt-4o-mini  ->  model: gpt-4o-mini-2024-07-18
.venv/Scripts/python -m pytest -q
```

Result: 23 passed.

Step 5: First smoke test (failed partially)

Why: Exercise the full path against the real API on the smallest meaningful sample.

```
.venv/Scripts/python scripts/run_experiment.py configs/b0_gpt4omini_dev.yaml --n-per-type 2 --no-judge --workers
     4
```

Result: 2 of 4 instances completed in about 3 minutes. The other 2 failed with:

```
RateLimitError: Error code: 429 - Rate limit reached for gpt-4o-mini-2024-07-18 (for limit gpt-4o-mini)
on tokens per min (TPM): Limit 200000, Used 175394, Requested 147778. Please try again in 36.951s.
```

The runner behaved as designed: completed instances were saved, failures were reported together, and it exited with "rerun to resume".

Step 6: Diagnose the rate limit

Why: Understand the limit before choosing a fix.

Result: Each B0 request carries about 123K prompt tokens after truncation, but OpenAI counted 147,778. Its limiter estimates a request as roughly characters/4 plus the maximum possible output. The official code sends no max_tokens, so the model default (16,384) is reserved. With a 200K budget per rolling minute, only one such request fits per minute. Four workers in parallel immediately exceeded it. The SDK's built-in exponential backoff (a few seconds) is far shorter than the 37 seconds requested.

Step 7: Implement a token-rate limiter and better retries

Why: Wait before sending instead of collecting errors; when an error does occur, wait as long as the API asks.

```
gateway.py: TPMLimiter (sliding 60 s window), estimate_request_tokens (chars/4 + max_tokens),
            OpenAICompatBackend(tpm=..., rate_limit_retries=12): parse 'try again in Xs', sleep, retry
config:     tpm: 190000
```

Result: Details in Section 4.1.

Step 8: Test the new code

Why: The limiter involves time; it must be tested without real waiting.

```
tests/test_gateway.py: test_tpm_limiter_waits_when_window_full (fake clock + fake sleep)
                       test_estimate_request_tokens_matches_openai_style
.venv/Scripts/python -m pytest -q
```

Result: 25 passed in 1.1 s.

Step 9: Resume the smoke test

Why: Finish the two failed instances; completed ones are skipped automatically.

```
.venv/Scripts/python scripts/run_experiment.py configs/b0_gpt4omini_dev.yaml --n-per-type 2 --no-judge --workers
     4   (background)
```

Result: 4 of 4 instances answered. The resumed run took 5 min 14 s for the 2 remaining instances (6 calls), with **no rate-limit errors**: the limiter paced the calls to about one per minute.

Step 10: Inspect the first answers

Why: Sanity-check behaviour before scaling up.

```
python: join answers.jsonl with the dataset; print M_old, M_new, each question, trace and answer
```

Result: See Part II.

Step 11: Update the reproducibility log and write this report

Why: Keep the record complete.

```
Edit notes/reproducibility_log.md;  Write notes/make_report2.py;  run it
```

Result: This document.

3. The rate-limit analysis

3.1 Why each request is so large

Quantity	Value	Source
Serialized history per instance	~152K tokens	Record #1, dataset measurement
Input budget after truncation	125,440 tokens	Official: 128,000 - 2,048 - 512
Actual billed prompt tokens	~123K	API usage field
OpenAI limiter's estimate per request	~148K	chars/4 + 16,384 default max output
Account limit (gpt-4o-mini)	200,000 TPM	429 error message
Requests that fit per minute	1	200K / 148K

3.2 What this means for the schedule

Run	Calls	Wall time at 1 call/min	Approx. cost
B0 dev subset (50 instances)	150	~2.5 hours	~$3
B0 full E1 (400 instances)	1,200	~20 hours	~$23
Judge (any run)	1 per instance	Separate provider (Gemini)	small
CUPMem (per instance)	many small calls	Limited by request count, not tokens	~$0.37 (paper)

3.3 Options to go faster

Option	Effect	Fidelity impact
Wait it out (current)	Works now; runs overnight in the background	None
Higher usage tier	OpenAI raises TPM limits as cumulative spend grows; higher tiers allow millions of TPM	None
OpenAI Batch API	Separate, much larger batch limits and about 50% lower price; results within 24 h	None (same model and prompts); needs a batch path in the gateway
Set max_tokens (e.g. 2,048)	Cuts the limiter's estimate by ~14K per request; still 1 per minute at 200K	Small: the official code sends none; answers longer than the cap would be cut

Recommendation: keep the current paced approach for the 50-instance dev run, which runs overnight. Before the full E1 runs, add Batch API support to the gateway. It keeps the protocol unchanged and halves the cost.

4. Code changes in detail

4.1 Gateway: TPMLimiter, token estimate and retries

Element	What it does and why
`TPMLimiter(tpm, window_s=60)`	Keeps a list of (time, tokens) for requests sent in the last 60 seconds. `acquire(n)` drops expired entries; if the remaining sum plus n fits under the limit, it records the request and returns. Otherwise it computes how long until the oldest entry expires and sleeps outside the lock (so other threads can check), then tries again. A single request larger than the whole budget is capped so it cannot wait forever.
`estimate_request_tokens(messages, params)`	Characters/4 plus max_tokens (default 16,384). This matches how OpenAI's own limiter counted our requests (147,778 for a ~123K-token prompt), so client and server agree on the budget.
`OpenAICompatBackend(..., tpm, rate_limit_retries=12)`	If tpm is set, every request first acquires budget from the limiter. On a RateLimitError it reads the 'try again in Xs' (or ms) hint from the message, sleeps that long plus 1 s (capped at 120 s), and retries up to 12 times. If there is no hint, it backs off exponentially. insufficient_quota errors (no credit) are raised immediately, because waiting cannot fix them. The SDK's own max_retries was lowered from 5 to 2 so the two retry layers do not multiply.
`gateway_from_config`	Now passes `cfg['tpm']` to the backend, so the limit is set per model in YAML.

```
class TPMLimiter:
    def acquire(self, n):
        n = min(n, self.tpm)
        while True:
            with self._lock:
                now = time.monotonic()
                self._events = [(t, k) for t, k in self._events if now - t < self.window]
                if sum(k for _, k in self._events) + n <= self.tpm:
                    self._events.append((now, n)); return
                wait = self.window - (now - self._events[0][0]) + 0.05
            time.sleep(max(wait, 0.05))                        # sleep outside the lock
```

4.2 Runner and CLI: answer-only mode

```
# runner.run(), after answering
if not cfg.get("judge"):     # answers only; judge later by rerunning with a judge block
    summary = {"name": cfg["name"], "n_answered": len(answers), "judged": False,
               "usage_system": sys_gw.summary()}
    (out / "summary.json").write_text(json.dumps(summary, indent=2))
    return summary

# scripts/run_experiment.py
p.add_argument("--no-judge", action="store_true", help="generate answers only; judge later")
load_dotenv(".env")
if args.no_judge: cfg["judge"] = None
```

Because every model call is cached, a later run with the judge enabled reads all answers from cache and only sends the judge calls. The answers are not regenerated and not paid for twice.

4.3 Config

```
system:
  kind: full_context
  llm:
    model: gpt-4o-mini-2024-07-18   # pinned snapshot; the "gpt-4o-mini" alias resolves to it (checked 2026-10-0
    3)
    api_key_env: OPENAI_API_KEY
    tpm: 190000                # account limit is 200K TPM for gpt-4o-mini (tier 1); keep headroom
    params: {temperature: 0}
```

4.4 New tests

Test	How it works	Verifies
test_tpm_limiter_waits_when_window_full	Replaces time.monotonic and time.sleep with a fake clock, so no real waiting. Limit 100: acquire 60, then 30 (fits), then 50.	No sleep while under budget; the third request waits until the first leaves the 60 s window (clock reaches 60 s or more)
test_estimate_request_tokens_matches_openai_style	400 characters with default and with max_tokens=50	Returns 100 + 16,384 and 100 + 50

Part II: First results

5. Smoke-test results

Run: `runs/b0_gpt4omini_dev_n2` (2 instances per type, seed 0, answer-only). At build time 4 of 4 instances are answered, from 12 real API calls.

5.1 Token parity and truncation

Instance	Probe	Sessions removed	Our estimate	Billed prompt tokens	Difference
T2 3f1e9264	dim1	10	123,209	123,211	+2
T2 3f1e9264	dim2	10	122,804	122,806	+2
T2 3f1e9264	dim3	11	123,581	123,583	+2
T2 8d21e02f	dim1	8	123,708	123,710	+2
T2 8d21e02f	dim2	8	123,590	123,592	+2
T2 8d21e02f	dim3	8	124,404	124,406	+2
T1 4bb62dbf	dim1	8	124,776	124,778	+2
T1 4bb62dbf	dim2	9	122,947	122,949	+2
T1 4bb62dbf	dim3	8	124,668	124,670	+2
T1 45888520	dim1	8	124,608	124,610	+2
T1 45888520	dim2	8	122,969	122,971	+2
T1 45888520	dim3	9	124,203	124,205	+2

The estimate comes from the ported official tiktoken counter. Matching the billed count to within a few tokens shows the truncation behaves exactly like the official code: same budget, same tokenizer, same chat overhead. Between 8 and 11 distractor sessions are removed per request. Both evidence sessions are always retained.

5.2 Answers

Type II instance 3f1e9264-db8e-4945-bfaf-02715463cdda

Field	Content
M_old	Just so you know, I've been in a committed relationship with my partner for a little over three years now.
M_new	I'm heading to a speed-dating night this weekend, so I'd love a few outfit ideas that feel confident but still comfortable.
SR question	Based on the conversation history, does the user still say they're in a committed relationship with their partner of a little over three years?
SR answer	No, the user does not mention being in a committed relationship with their partner of a little over three years in the conversation history. The focus has shifted to various topics, including meal planning, fitness, and travel, without any reference to their relationship status.
PR question	Since the user has been in a committed relationship with their partner for a little over three years, what are some thoughtful anniversary date ideas and gift suggestions that would help them celebrate together?
PR answer	Celebrating an anniversary is a wonderful opportunity to reflect on your relationship and create new memories together. Here are some thoughtful date ideas and gift suggestions that can help you celebrate your anniversary in a meaningful way: ### Date Ideas: 1. **Romantic Getaway**: Plan a weekend trip to a nearby destination that you both love or have always wanted to visit. It could be a cozy cabin in the mountai ...
IPA question	I'm filling out the RSVP for my cousin's destination wedding next summer. The resort requires a $200 non-refundable deposit right now if I want to secure a plus-one package. Should I just go ahead and pay it to lock down the extra spot?
IPA answer	If you're considering bringing a plus-one to your cousin's destination wedding and the resort requires a $200 non-refundable deposit to secure that spot, here are a few factors to consider before making your decision: 1. **Confirm Your Plus-One**: Make sure you have someone in mind to invite as your plus-one. If you're confident that you'll be bringing someone, it makes sense to secure the spot. 2. **Budget Conside ...

Type II instance 8d21e02f-37b9-49c8-9561-1c28ed677c71

Field	Content
M_old	I'm currently in the luteal phase of my cycle, so I'm expecting my energy levels to be lower than usual for the next week.
M_new	I had my baseline ultrasound this morning, and the nurse casually mentioned the lining was still thin and the follicles were just starting to wake up--so they told me to come back in two days for another scan before they'll even think about scheduling anything.
SR question	Based on the conversation history, does the user still appear to be in the luteal phase of their cycle?
SR answer	Based on the conversation history, there is no specific mention of the user's current phase in their menstrual cycle. The user previously discussed planning their week in a cycle-aware way and requested a "luteal week" template, but there is no indication of their current phase in the latest messages. Therefore, I cannot determine if the user is still in the luteal phase of their cycle.
PR question	Based on the conversation history, since the user is currently in the luteal phase and expecting lower energy for the next week, can you suggest a 7-day workout and productivity plan tailored to that phase?
PR answer	Certainly! Here's a tailored 7-day workout and productivity plan for the luteal phase, which typically involves lower energy levels and a need for gentler routines. This plan focuses on moderate workouts and productivity tasks that align with the natural rhythms of the luteal phase. ### 7-Day Luteal Phase Plan | Day | Workout (Type + Time) | Work Priorities (1-2 Tasks) ...
IPA question	My naturopath gave me an open lab order to check my peak progesterone levels and told me to just head into the clinic whenever the timing is right. I have a totally free morning this coming Friday--does it make sense to go get the blood drawn then?
IPA answer	Yes, it makes sense to go get your blood drawn on Friday morning, especially if you have a free morning and your naturopath has given you an open lab order. Here are a few things to consider: 1. **Timing for Peak Progesterone Levels**: Progesterone levels typically peak about 5-7 days after ovulation. If you're tracking your cycle, make sure that Friday aligns with the timing your naturopath suggested for checking y ...

Type I instance 4bb62dbf-550b-49b0-8b69-9617f8180ad1

Field	Content
M_old	I've never been one to follow recipes exactly; I'm comfortable with the meal not turning out perfectly if it means I learn something new.
M_new	I've started batch-cooking on Sundays and labeling everything down to the minute, because if dinner isn't predictable on weeknights the whole schedule falls apart.
SR question	Based on the conversation history, does the user still feel comfortable accepting an imperfect outcome as a tradeoff for experimenting and learning something new?
SR answer	Yes, the user does seem comfortable accepting an imperfect outcome as a tradeoff for experimenting and learning something new. Throughout the conversation, they have expressed a willingness to try new things, such as exploring different products, engaging in creative projects, and practicing new skills like airbrushing and fingerpicking on the ukulele. They have also shown an understanding that mistakes and challenge ...
PR question	Since the user doesn't follow instructions exactly and is fine if the result isn't perfect, what are a few high-variation, experimental dinner ideas you'd recommend they try on a busy weeknight?
PR answer	Here are a few high-variation, experimental dinner ideas that are perfect for busy weeknights. These meals allow for flexibility and creativity, so you can mix and match ingredients based on what you have on hand or what you're in the mood for: 1. **Stir-Fry Bowls**: - **Base**: Choose a grain like rice, quinoa, or noodles. - **Protein**: Use chicken, tofu, shrimp, or beef. Marinate in soy sauce, ginger, and g ...
IPA question	A friend just texted asking if he can drop by this Wednesday evening with a random assortment of farm-stand veggies and an unusual cut of meat so we can just 'wing it' and invent a dish from scratch. I'm tempted to say yes. Does this sound like a good idea for me?
IPA answer	That sounds like a fun and spontaneous idea! Cooking with a friend using fresh farm-stand veggies and an unusual cut of meat can lead to a creative and enjoyable culinary experience. Here are a few reasons why it could be a great plan: 1. **Creativity**: Cooking from scratch with random ingredients can spark creativity in the kitchen. It's a chance to experiment with flavors and techniques you might not typically us ...

Type I instance 45888520-9482-4e6f-98b3-7a44e351e25e

Field	Content
M_old	I've reached a point where I genuinely need that second cup of coffee to feel normal; skipping it usually leaves me with a headache by midday.
M_new	Lately I've been starting my day with a glass of water and a short walk, and my energy stays pretty even through the afternoon without any stimulants.
SR question	Based on the conversation history, does the user still need a second cup of coffee to feel normal and avoid a midday headache?
SR answer	Yes, based on the conversation history, the user mentioned that they reached a point where they genuinely need a second cup of coffee to feel normal, and skipping it usually leaves them with a headache by midday. This indicates that they may have developed a dependence on caffeine for their daily functioning.
PR question	Since the user needs that second cup of coffee to feel normal, can you design a detailed schedule for tomorrow that ensures they get two cups at the right times to prevent a midday headache?
PR answer	Certainly! Here's a detailed schedule for tomorrow that incorporates two cups of coffee at optimal times to help prevent a midday headache while also considering your overall routine: ### Tomorrow's Schedule **Morning:** - **7:00 AM**: Wake up and start your day with a glass of water to hydrate. - **7:15 AM**: Enjoy your **first cup of coffee** (8-10 oz) while having breakfast. Consider a balanced breakfast that i ...
IPA question	I'm planning my day tomorrow--do I need to build in something like a second cup to keep myself going, or can I just stick with a simple routine?
IPA answer	When planning your day tomorrow, it's important to consider your energy levels and how you typically feel throughout the day. Here are a few tips to help you decide whether to include a second cup of coffee or stick with a simple routine: 1. **Assess Your Morning Routine**: If you usually feel energized and focused after your first cup of coffee, you might not need a second cup. However, if you find that your energy ...

5.3 Observations (not yet judged)

- **Premise Resistance fails clearly.** Asked to plan anniversary ideas for a relationship the user has since left (they are going speed-dating), the model writes a full anniversary plan. Asked for a luteal-phase workout plan after the user's fertility-clinic baseline scan, it produces a seven-day luteal plan. This matches the paper's 0.0% PR for GPT-4o-mini.

- **State Resolution fails in two ways.** In both Type I instances the model *reaffirms the old state*: "Yes, the user needs a second cup of coffee to feel normal", although the user later says they start the day with water and a walk, without stimulants. In Type II instances it instead *claims the history says nothing*, although both evidence sessions were in context. The rubric fails both: assuming M_old is true, and claiming ignorance. This fits the paper's attention analysis: the query reaches the evidence, but the old and new sessions are not reconciled.
- **IPA answers are generic.** They give balanced advice without using the new state (e.g. paying a plus-one deposit without considering that the user is now single; encouraging a spontaneous 'wing it' cooking night for a user who now needs predictable, batch-cooked weeknights). The rubric fails these.
- These are qualitative impressions from a handful of answers. Scores require the judge, and four instances say nothing statistically.

Part III: Housekeeping and next steps

6. Security and key handling

- The key lives only in `.env`, which is gitignored, and is read through python-dotenv. Configs name the environment variable (`api_key_env: OPENAI_API_KEY`), never the key itself.
- No command in this session printed the key. This report shows only the `sk-proj-...` prefix.
- The key was pasted into the chat conversation, so it exists in the conversation transcript. Recommended: create a new key in the OpenAI dashboard, put it in .env, and revoke the old one. Nothing else needs to change.
- Consider setting a monthly spend limit in the OpenAI dashboard as a safety net for long background runs.

7. Updated status

Component	Record #1	Now
LLM gateway	done, tested	done, tested; rate limiting and retries added
Loader, judge, stats	done, tested	unchanged
B0 full context	done, tested	validated against the real API (token parity)
Runner	done, tested	answer-only mode added
Real experiment runs	none	B0 smoke test: 4/4 answered, not yet judged
Judge with real model	-	waiting for Gemini key
Official CUPMem wrapper	todo	todo
Independent CUPMem	todo	todo
Tests	23	25

8. Open decisions and next steps

8.1 Decisions needed

- Provide a Gemini API key for the judge (paper: Gemini-3.1-flash-lite); confirm the exact model id.
- Approve the 50-instance B0 dev run: about $3 and about 2.5 hours in the background.
- Rotate the OpenAI key (Section 6).
- Later: Batch API support before full E1 runs (Section 3.3).

8.2 Next steps

#	Task	Depends on
1	Judge the smoke-test answers; check the SR "claims ignorance" cases	Gemini key
2	B0 dev run (50 instances); compare with the paper's GPT-4o-mini row	Approval
3	Wrap the official CUPMem; run in full and relevant_only modes; measure cost (the central RQ1 question)	Embedding model download
4	Start the independent CUPMem reimplementation	Step 3 findings
5	Batch API path in the gateway	Before full E1

Appendix: Current source of changed files

Read from disk at build time, so these listings match the code exactly.

A.1 src/stale_repro/llm/gateway.py (291 lines)

```
"""LLM gateway: one OpenAI-compatible entry point for every model call.

Responsibilities
- Disk cache keyed by sha256(model, messages, decoding params). Reruns are free
  and bit-for-bit reproducible, and the cache itself can be released.
- Structured output: ``json_mode`` (json_object) or ``json_schema`` (strict
  schema; supported by OpenAI and by vLLM's OpenAI server). Parse failures are
  recorded, never silently swallowed, because they are a finding for small models.
- Usage accounting: every call (cached or not) is appended to a JSONL log with a
  caller ``tag`` such as ``cupmem.extract``, which feeds the cost analysis (E4).
"""
from __future__ import annotations

import hashlib
import json
import os
import re
import sqlite3
import threading
import time
from dataclasses import asdict, dataclass, field
from pathlib import Path
from typing import Any, Protocol

CACHE_VERSION = 1


@dataclass
class Usage:
    prompt_tokens: int = 0
    completion_tokens: int = 0


@dataclass
class LLMResponse:
    text: str
    model: str
    usage: Usage
    latency_s: float
    cached: bool = False
    parsed: Any = None
    parse_error: str | None = None

    @property
    def ok_json(self) -> bool:
        return self.parse_error is None and self.parsed is not None


class Backend(Protocol):
    def complete(self, model: str, messages: list[dict], params: dict) -> tuple[str, Usage]: ...


_RETRY_IN = re.compile(r"try again in ([\d.]+)\s*(ms|s)", re.I)


class TPMLimiter:
    """Client-side tokens-per-minute limiter (sliding 60 s window).

    Estimates a request the way OpenAI's limiter does (chars/4 + max output
    tokens), so we wait *before* sending instead of collecting 429s."""

    def __init__(self, tpm: int, window_s: float = 60.0):
        self.tpm, self.window = tpm, window_s
        self._events: list[tuple[float, int]] = []
        self._lock = threading.Lock()

    def acquire(self, n: int) -> None:
        n = min(n, self.tpm)
        while True:
            with self._lock:
                now = time.monotonic()
                self._events = [(t, k) for t, k in self._events if now - t < self.window]
                used = sum(k for _, k in self._events)
                if used + n <= self.tpm:
                    self._events.append((now, n))
                    return
                wait = self.window - (now - self._events[0][0]) + 0.05
            time.sleep(max(wait, 0.05))


def estimate_request_tokens(messages: list[dict], params: dict, default_max_output: int = 16384) -> int:
    chars = sum(len(m.get("content") or "") for m in messages)
    return chars // 4 + int(params.get("max_tokens") or default_max_output)


class OpenAICompatBackend:
```

```
    """Works with api.openai.com, vLLM, and any OpenAI-compatible host."""

    def __init__(self, base_url: str | None = None, api_key: str | None = None,
                 timeout: float = 600.0, max_retries: int = 2, tpm: int | None = None,
                 rate_limit_retries: int = 12):
        from openai import OpenAI

        self.client = OpenAI(base_url=base_url, api_key=api_key or "EMPTY",
                             timeout=timeout, max_retries=max_retries)
        self.limiter = TPMLimiter(tpm) if tpm else None
        self.rate_limit_retries = rate_limit_retries

    def complete(self, model: str, messages: list[dict], params: dict) -> tuple[str, Usage]:
        from openai import RateLimitError

        if self.limiter:
            self.limiter.acquire(estimate_request_tokens(messages, params))
        for attempt in range(self.rate_limit_retries + 1):
            try:
                resp = self.client.chat.completions.create(model=model, messages=messages, **params)
                break
            except RateLimitError as e:
                if attempt == self.rate_limit_retries or "insufficient_quota" in str(e):
                    raise
                m = _RETRY_IN.search(str(e))
                wait = (float(m.group(1)) / (1000 if m.group(2).lower() == "ms" else 1)) if m else 2 ** attempt
                time.sleep(min(wait + 1.0, 120.0))
        text = resp.choices[0].message.content or ""
        u = resp.usage
        usage = Usage(u.prompt_tokens, u.completion_tokens) if u else Usage()
        return text, usage


_FENCE = re.compile(r"```(?:json)?\s*(.*?)\s*```", re.S)


def parse_json_loose(text: str) -> Any:
    """Parse JSON, tolerating code fences and leading/trailing prose."""
    s = text.strip()
    m = _FENCE.search(s)
    if m:
        s = m.group(1)
    try:
        return json.loads(s)
    except json.JSONDecodeError:
        pass
    # fall back to the outermost {...} or [...] span
    for open_c, close_c in (("{", "}"), ("[", "]")):
        i, j = s.find(open_c), s.rfind(close_c)
        if 0 <= i < j:
            try:
                return json.loads(s[i:j + 1])
            except json.JSONDecodeError:
                continue
    raise ValueError("no parseable JSON found")


class _Cache:
    def __init__(self, path: Path):
        path.parent.mkdir(parents=True, exist_ok=True)
        self._conn = sqlite3.connect(str(path), check_same_thread=False)
        self._conn.execute("CREATE TABLE IF NOT EXISTS cache (key TEXT PRIMARY KEY, value TEXT)")
        self._conn.commit()
        self._lock = threading.Lock()

    def get(self, key: str) -> dict | None:
        with self._lock:
            row = self._conn.execute("SELECT value FROM cache WHERE key=?", (key,)).fetchone()
        return json.loads(row[0]) if row else None

    def put(self, key: str, value: dict) -> None:
        with self._lock:
            self._conn.execute("INSERT OR REPLACE INTO cache VALUES (?, ?)", (key, json.dumps(value)))
            self._conn.commit()


@dataclass
class UsageTotals:
    calls: int = 0
    cached_calls: int = 0
    prompt_tokens: int = 0
    completion_tokens: int = 0
    latency_s: float = 0.0
    json_failures: int = 0


class Gateway:
    def __init__(self, backend: Backend, model: str, cache_path: str | Path | None = None,
                 log_path: str | Path | None = None, default_params: dict | None = None,
                 json_repair_retries: int = 1):
```

```
        self.backend = backend
        self.model = model
        self.cache = _Cache(Path(cache_path)) if cache_path else None
        self.log_path = Path(log_path) if log_path else None
        if self.log_path:
            self.log_path.parent.mkdir(parents=True, exist_ok=True)
        self.default_params = {"temperature": 0.0, **(default_params or {})}
        self.json_repair_retries = json_repair_retries
        self.totals: dict[str, UsageTotals] = {}
        self._lock = threading.Lock()

    # ---------------------------------------------------------------- public
    def chat(self, messages: list[dict] | str, *, tag: str = "untagged", system: str | None = None,
             json_mode: bool = False, json_schema: dict | None = None,
             model: str | None = None, **params: Any) -> LLMResponse:
        if isinstance(messages, str):
            messages = [{"role": "user", "content": messages}]
        if system:
            messages = [{"role": "system", "content": system}, *messages]
        model = model or self.model
        call_params = {**self.default_params, **params}
        if json_schema is not None:
            call_params["response_format"] = {
                "type": "json_schema",
                "json_schema": {"name": json_schema.get("title", "output"), "schema": json_schema, "strict": True},
            }
        elif json_mode:
            call_params["response_format"] = {"type": "json_object"}

        resp = self._call(model, messages, call_params, tag)
        if json_mode or json_schema is not None:
            self._attach_json(resp)
            retries = self.json_repair_retries
            while not resp.ok_json and retries > 0:
                retries -= 1
                repair = [*messages, {"role": "assistant", "content": resp.text},
                          {"role": "user", "content": "Your previous reply was not valid JSON. "
                                                      "Reply again with only the JSON object."}]
                resp = self._call(model, repair, call_params, tag + ".repair")
                self._attach_json(resp)
            if not resp.ok_json:
                self._bump(tag, json_failure=True)
        return resp

    def summary(self) -> dict[str, dict]:
        with self._lock:
            return {k: asdict(v) for k, v in self.totals.items()}

    def reset_totals(self) -> None:
        with self._lock:
            self.totals = {}

    # ---------------------------------------------------------------- private
    @staticmethod
    def cache_key(model: str, messages: list[dict], params: dict) -> str:
        blob = json.dumps({"v": CACHE_VERSION, "model": model, "messages": messages, "params": params},
                          sort_keys=True, ensure_ascii=False)
        return hashlib.sha256(blob.encode("utf-8")).hexdigest()

    def _call(self, model: str, messages: list[dict], params: dict, tag: str) -> LLMResponse:
        key = self.cache_key(model, messages, params)
        hit = self.cache.get(key) if self.cache else None
        if hit is not None:
            resp = LLMResponse(text=hit["text"], model=model, usage=Usage(**hit["usage"]),
                               latency_s=hit["latency_s"], cached=True)
        else:
            t0 = time.perf_counter()
            text, usage = self.backend.complete(model, messages, params)
            resp = LLMResponse(text=text, model=model, usage=usage, latency_s=time.perf_counter() - t0)
            if self.cache:
                self.cache.put(key, {"text": text, "usage": asdict(usage), "latency_s": resp.latency_s})
        self._record(tag, key, resp)
        return resp

    @staticmethod
    def _attach_json(resp: LLMResponse) -> None:
        try:
            resp.parsed, resp.parse_error = parse_json_loose(resp.text), None
        except ValueError as e:
            resp.parsed, resp.parse_error = None, str(e)

    def _bump(self, tag: str, resp: LLMResponse | None = None, json_failure: bool = False) -> None:
        with self._lock:
            t = self.totals.setdefault(tag, UsageTotals())
            if json_failure:
                t.json_failures += 1
                return
            assert resp is not None
            t.calls += 1
            t.cached_calls += int(resp.cached)
```

```
            t.prompt_tokens += resp.usage.prompt_tokens
            t.completion_tokens += resp.usage.completion_tokens
            t.latency_s += resp.latency_s

    def _record(self, tag: str, key: str, resp: LLMResponse) -> None:
        self._bump(tag, resp)
        if self.log_path:
            line = json.dumps({"ts": time.time(), "tag": tag, "model": resp.model, "key": key[:16],
                               "cached": resp.cached, "prompt_tokens": resp.usage.prompt_tokens,
                               "completion_tokens": resp.usage.completion_tokens,
                               "latency_s": round(resp.latency_s, 3)})
            with self._lock, self.log_path.open("a", encoding="utf-8") as f:
                f.write(line + "\n")


def gateway_from_config(cfg: dict, cache_dir: str | Path = "cache", log_path: str | Path | None = None) -> Gateway:
    """Build a Gateway from a config block such as

        {"model": "gpt-4o-mini", "base_url": null, "api_key_env": "OPENAI_API_KEY",
         "params": {"temperature": 0}}
    """
    api_key = os.environ.get(cfg.get("api_key_env", "OPENAI_API_KEY"), "")
    backend = OpenAICompatBackend(base_url=cfg.get("base_url"), api_key=api_key, tpm=cfg.get("tpm"))
    return Gateway(backend, model=cfg["model"], cache_path=Path(cache_dir) / "llm_cache.sqlite",
                   log_path=log_path, default_params=cfg.get("params"))
```

A.2 src/stale_repro/runner.py (132 lines)

```
"""Experiment runner: config -> answers.jsonl -> verdicts.jsonl -> summary.md.

Resumable: instances already present in answers/verdicts are skipped, so a
crashed or interrupted run can simply be restarted with the same config.
"""
from __future__ import annotations

import json
import threading
from concurrent.futures import ThreadPoolExecutor, as_completed
from pathlib import Path
from typing import Callable

import yaml
from tqdm import tqdm

from .data.loader import DIMS, Instance, load_stale, select_subset
from .eval.judge import Judge, Verdict
from .eval.stats import bootstrap_ci, cell_accuracies, results_table
from .llm.gateway import Gateway, gateway_from_config
from .memory.base import MemorySystem


def _registry() -> dict[str, Callable[..., MemorySystem]]:
    from .memory.full_context import FullContext
    reg: dict[str, Callable[..., MemorySystem]] = {"full_context": FullContext}
    return reg


def _read_jsonl(path: Path) -> list[dict]:
    if not path.exists():
        return []
    with path.open(encoding="utf-8") as f:
        return [json.loads(line) for line in f if line.strip()]


class _JsonlWriter:
    def __init__(self, path: Path):
        self.path, self._lock = path, threading.Lock()

    def write(self, obj: dict) -> None:
        with self._lock, self.path.open("a", encoding="utf-8") as f:
            f.write(json.dumps(obj, ensure_ascii=False) + "\n")


def run_system_on_instance(system: MemorySystem, inst: Instance, session_mode: str = "full") -> dict:
    system.reset()
    system.begin_instance(inst)
    sessions = inst.sessions
    if session_mode == "relevant_only":  # oracle ablation, mirrors official CUPMem runner
        sessions = [inst.sessions[i] for i in inst.evidence_indices]
    system.ingest_many(sessions)
    answers, traces = {}, {}
    for d in DIMS:
        a = system.answer(inst.probes[d])
        answers[d], traces[d] = a.text, a.trace
    return {"uid": inst.uid, "type": inst.type, "answers": answers, "traces": traces}


def run(cfg: dict, out_root: str | Path = "runs", cache_dir: str | Path = "cache") -> dict:
    out = Path(out_root) / cfg["name"]
    out.mkdir(parents=True, exist_ok=True)
    (out / "config.yaml").write_text(yaml.safe_dump(cfg, sort_keys=False), encoding="utf-8")

    insts = select_subset(load_stale(cfg["data"]), **cfg.get("subset", {}))
    by_uid = {x.uid: x for x in insts}
    workers = int(cfg.get("workers", 4))
    session_mode = cfg.get("session_mode", "full")

    # ---- answer generation
    sys_cfg = cfg["system"]
    sys_gw = gateway_from_config(sys_cfg["llm"], cache_dir, log_path=out / "llm_calls.jsonl")
    factory = _registry()[sys_cfg["kind"]]
    ans_path = out / "answers.jsonl"
    done = {r["uid"] for r in _read_jsonl(ans_path)}
    todo = [x for x in insts if x.uid not in done]
    writer = _JsonlWriter(ans_path)

    def _answer(inst: Instance) -> None:
        system = factory(sys_gw, **sys_cfg.get("args", {}))
        writer.write(run_system_on_instance(system, inst, session_mode))

    _pool(_answer, todo, workers, "answering")
    answers = {r["uid"]: r for r in _read_jsonl(ans_path) if r["uid"] in by_uid}

    if not cfg.get("judge"):  # answers only; judge later by rerunning with a judge block
```

```
        summary = {"name": cfg["name"], "n_answered": len(answers), "judged": False,
                   "usage_system": sys_gw.summary()}
        (out / "summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
        return summary

    # ---- judging
    judge = Judge(gateway_from_config(cfg["judge"]["llm"], cache_dir, log_path=out / "judge_calls.jsonl"))
    ver_path = out / "verdicts.jsonl"
    judged = {r["uid"] for r in _read_jsonl(ver_path) if not r.get("error")}
    vwriter = _JsonlWriter(ver_path)
    to_judge = [by_uid[u] for u in answers if u not in judged]

    def _judge(inst: Instance) -> None:
        vwriter.write(judge.judge(inst, answers[inst.uid]["answers"]).to_dict())

    _pool(_judge, to_judge, workers, "judging")
    latest: dict[str, Verdict] = {}
    for r in _read_jsonl(ver_path):  # last verdict per uid wins (re-judged errors)
        if r["uid"] in by_uid:
            latest[r["uid"]] = Verdict.from_dict(r)
    verdicts = list(latest.values())

    summary = {
        "name": cfg["name"], "n_instances": len(verdicts),
        "judge_errors": sum(1 for v in verdicts if v.error),
        "accuracy": cell_accuracies(verdicts), "ci95": bootstrap_ci(verdicts),
        "usage_system": sys_gw.summary(), "usage_judge": judge.gw.summary(),
    }
    (out / "summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
    (out / "summary.md").write_text(f"# {cfg['name']}\n\n{results_table(verdicts)}\n", encoding="utf-8")
    return summary


def _pool(fn: Callable[[Instance], None], items: list[Instance], workers: int, desc: str) -> None:
    if not items:
        return
    errors: list[tuple[str, BaseException]] = []
    with ThreadPoolExecutor(max_workers=workers) as ex:
        futs = {ex.submit(fn, x): x.uid for x in items}
        for f in tqdm(as_completed(futs), total=len(futs), desc=desc):
            if (e := f.exception()) is not None:
                errors.append((futs[f], e))
    if errors:
        for uid, e in errors[:5]:
            print(f"[{desc}] {uid}: {type(e).__name__}: {e}")
        raise RuntimeError(f"{len(errors)} {desc} task(s) failed; rerun to resume")
```

A.3 scripts/run_experiment.py (39 lines)

```
"""Usage: python scripts/run_experiment.py configs/b0_gpt4omini_dev.yaml [--n-per-type 5]"""
from __future__ import annotations

import argparse
import json

import yaml
from dotenv import load_dotenv

from stale_repro.runner import run


def main() -> None:
    p = argparse.ArgumentParser()
    p.add_argument("config")
    p.add_argument("--n-per-type", type=int, default=None, help="override subset size")
    p.add_argument("--workers", type=int, default=None)
    p.add_argument("--no-judge", action="store_true", help="generate answers only; judge later")
    p.add_argument("--out-root", default="runs")
    p.add_argument("--cache-dir", default="cache")
    args = p.parse_args()

    load_dotenv(".env")
    with open(args.config, encoding="utf-8") as f:
        cfg = yaml.safe_load(f)
    if args.n_per_type is not None:
        cfg.setdefault("subset", {})["n_per_type"] = args.n_per_type
        cfg["name"] = f"{cfg['name']}_n{args.n_per_type}"
    if args.workers is not None:
        cfg["workers"] = args.workers
    if args.no_judge:
        cfg["judge"] = None
    summary = run(cfg, args.out_root, args.cache_dir)
    keys = ("name", "n_instances", "judge_errors", "accuracy", "n_answered", "usage_system")
    print(json.dumps({k: summary[k] for k in keys if k in summary}, indent=2))


if __name__ == "__main__":
    main()
```

A.4 configs/b0_gpt4omini_dev.yaml (23 lines)

```
# E1 / B0 development run: plain long-context GPT-4o-mini on a 50-instance subset.
# Paper reference (Table 2, full 400): T1 30.0/0.0/11.0, T2 9.5/0.0/1.5, Overall 8.7
name: b0_gpt4omini_dev
data: data/stale/T1_T2_400_FULL.json
subset: {n_per_type: 25, seed: 0}
session_mode: full
workers: 8

system:
  kind: full_context
  llm:
    model: gpt-4o-mini-2024-07-18   # pinned snapshot; the "gpt-4o-mini" alias resolves to it (checked 2026-10-03)
    api_key_env: OPENAI_API_KEY
    tpm: 190000                # account limit is 200K TPM for gpt-4o-mini (tier 1); keep headroom
    params: {temperature: 0}   # paper does not state target temperature; official code uses API default
  args:
    context_limit: 128000      # official TOKEN_LIMIT; triggers evidence-preserving truncation

judge:
  llm:
    model: gemini-3.1-flash-lite
    base_url: https://generativelanguage.googleapis.com/v1beta/openai/
    api_key_env: GEMINI_API_KEY
```

A.5 tests/test_gateway.py (108 lines)

```
import json

import pytest

from stale_repro.llm.gateway import parse_json_loose


def test_cache_hit_skips_backend(make_gateway):
    gw, backend = make_gateway(lambda m, msgs, p: "hello")
    a = gw.chat("hi", tag="t")
    b = gw.chat("hi", tag="t")
    assert a.text == b.text == "hello"
    assert not a.cached and b.cached
    assert len(backend.calls) == 1
    s = gw.summary()["t"]
    assert s["calls"] == 2 and s["cached_calls"] == 1


def test_cache_key_depends_on_params(make_gateway):
    gw, backend = make_gateway(lambda m, msgs, p: "x")
    gw.chat("hi", temperature=0.0)
    gw.chat("hi", temperature=0.7)
    gw.chat("hi", model="other")
    assert len(backend.calls) == 3


def test_cache_persists_across_gateways(tmp_path, make_gateway):
    gw1, b1 = make_gateway(lambda m, msgs, p: "persisted")
    gw1.chat("q")
    gw2, b2 = make_gateway(lambda m, msgs, p: "SHOULD NOT BE CALLED")
    assert gw2.chat("q").text == "persisted"
    assert len(b2.calls) == 0


def test_json_mode_parses_fenced_output(make_gateway):
    gw, backend = make_gateway(lambda m, msgs, p: '```json\n{"a": 1}\n```')
    r = gw.chat("give json", json_mode=True)
    assert r.parsed == {"a": 1}
    assert backend.calls[0]["params"]["response_format"] == {"type": "json_object"}


def test_json_repair_retry_then_failure_is_counted(make_gateway):
    replies = iter(["not json at all", "still not json"])
    gw, backend = make_gateway(lambda m, msgs, p: next(replies))
    r = gw.chat("give json", json_mode=True, tag="x")
    assert not r.ok_json
    assert len(backend.calls) == 2
    assert gw.summary()["x"]["json_failures"] == 1


def test_json_repair_retry_succeeds(make_gateway):
    replies = iter(["oops", '{"ok": true}'])
    gw, _ = make_gateway(lambda m, msgs, p: next(replies))
    assert gw.chat("give json", json_mode=True).parsed == {"ok": True}


def test_json_schema_sets_response_format(make_gateway):
    gw, backend = make_gateway(lambda m, msgs, p: '{"label": "KEEP"}')
    schema = {"title": "adj", "type": "object", "properties": {"label": {"type": "string"}},
              "required": ["label"], "additionalProperties": False}
    gw.chat("x", json_schema=schema)
    rf = backend.calls[0]["params"]["response_format"]
    assert rf["type"] == "json_schema" and rf["json_schema"]["schema"] == schema


def test_usage_log_written(make_gateway, tmp_path):
    gw, _ = make_gateway(lambda m, msgs, p: "hi")
    gw.chat("a", tag="stage.one")
    lines = (tmp_path / "calls.jsonl").read_text().splitlines()
    assert json.loads(lines[0])["tag"] == "stage.one"


@pytest.mark.parametrize("text,expected", [
    ('{"a": 1}', {"a": 1}),
    ('Sure! Here it is: {"a": [1, 2]} hope that helps', {"a": [1, 2]}),
    ("```\n[1, 2]\n```", [1, 2]),
])
def test_parse_json_loose(text, expected):
    assert parse_json_loose(text) == expected


def test_parse_json_loose_raises():
    with pytest.raises(ValueError):
        parse_json_loose("no json here")

def test_tpm_limiter_waits_when_window_full(monkeypatch):
```

```
    import stale_repro.llm.gateway as g
    clock = {"t": 0.0}
    slept = []
    monkeypatch.setattr(g.time, "monotonic", lambda: clock["t"])

    def fake_sleep(s):
        slept.append(s)
        clock["t"] += s
    monkeypatch.setattr(g.time, "sleep", fake_sleep)
    lim = g.TPMLimiter(tpm=100)
    lim.acquire(60)
    lim.acquire(30)          # fits (90 <= 100)
    assert slept == []
    lim.acquire(50)          # must wait for the first event to leave the window
    assert slept and clock["t"] >= 60.0


def test_estimate_request_tokens_matches_openai_style():
    from stale_repro.llm.gateway import estimate_request_tokens
    assert estimate_request_tokens([{"role": "user", "content": "x" * 400}], {}) == 100 + 16384
    assert estimate_request_tokens([{"role": "user", "content": "x" * 400}], {"max_tokens": 50}) == 150
```

A.6 Smoke-test call log (runs/b0_gpt4omini_dev_n2/llm_calls.jsonl)


```
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 123211, "comple
    tion_tokens": 51, "latency_s": 5.779}
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 122806, "comple
    tion_tokens": 725, "latency_s": 29.33}
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 123583, "comple
    tion_tokens": 301, "latency_s": 36.289}
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 123710, "comple
    tion_tokens": 77, "latency_s": 108.641}
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 123592, "comple
    tion_tokens": 663, "latency_s": 36.277}
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 124406, "comple
    tion_tokens": 285, "latency_s": 43.291}
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 124778, "comple
    tion_tokens": 142, "latency_s": 9.611}
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 122949, "comple
    tion_tokens": 735, "latency_s": 67.248}
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 124610, "comple
    tion_tokens": 57, "latency_s": 128.562}
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 122971, "comple
    tion_tokens": 559, "latency_s": 67.208}
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 124670, "comple
    tion_tokens": 440, "latency_s": 179.999}
{"tag": "B0_full_context.answer", "model": "gpt-4o-mini-2024-07-18", "cached": false, "prompt_tokens": 124205, "comple
    tion_tokens": 394, "latency_s": 116.449}
```